



Carátula para entrega de prácticas

Facultad de Ingeniería

Laboratorios de docencia

Laboratorio de Computación Salas A y B

Profesor(a): René Adrián Dávila Pérez

Asignatura: Programación Orientada a Objetos

Grupo: 07

No de Prácticas(s): 04

Integrante(s): 323078401

323115296

323082392

323239441

323253916

No. de lista o

brigada: 04

Semestre: 2027-I

Fecha de entrega: 24/09/2026

Observaciones: _____

CALIFICACIÓN: _____

Índice

1. Introducción	2
1.1. Planteamiento del problema	2
1.2. Motivación	2
1.3. Objetivos	2
2. Marco Teórico	2
3. Desarrollo	3
3.1. Actividad 1: Análisis comparativo del concepto de destructor mediante LLM	3
3.1.1. Análisis de la respuesta 1	3
3.1.2. Análisis de la respuesta 2	3
3.1.3. Análisis de la respuesta 3	4
3.1.4. Observaciones	4
3.2. Actividad 2: Implementación del Analizador de Palabras	5
4. Resultados	6
5. Conclusiones	7

1. Introducción

1.1. Planteamiento del problema

En esta práctica se requiere procesar cadenas de caracteres para identificar patrones repetidos, ignorando variaciones de formato como mayúsculas, minúsculas o signos de puntuación. De forma paralela surge la necesidad de comprender cómo se gestiona la memoria y el ciclo de vida de las instancias dentro de Java, donde no existe un destructor formal como en otros lenguajes de programación.

1.2. Motivación

El procesamiento de datos textuales mediante estructuras de datos adecuadas para evitar iteraciones innecesarias o algoritmos de alta complejidad. Asimismo, comprender el funcionamiento del recolector de basura (Garbage Collector) y los mecanismos de liberación explícita de recursos para construir aplicaciones que eviten fugas de memoria.

1.3. Objetivos

- Analizar las respuestas emitidas por distintos modelos de lenguaje (LLM) respecto al concepto de destructor en Java.
- Desarrollar una aplicación que determine y contabilice la frecuencia de palabras duplicadas en una oración, ignorando puntuación y diferencias entre mayúsculas y minúsculas.
- Validar el funcionamiento del programa mediante pruebas de ejecución con entradas que incluyan caracteres especiales, acentos, la letra "ñ", números y combinaciones de mayúsculas y minúsculas.

2. Marco Teórico

La gestión de memoria en Java difiere del modelo tradicional de lenguajes como C++. En Java no existen los destructores explícitos; en su lugar, la Máquina Virtual de Java (JVM) emplea un entorno de administración automática conocido como recolector de basura (Garbage Collector) [1]. Este proceso secundario monitorea la memoria Heap e identifica aquellos objetos que han quedado inalcanzables al perder todas sus referencias activas, liberando la memoria ocupada de forma asíncrona e indeterminista [1].

Para la liberación de recursos externos tales como archivos o conexiones de red que no pueden quedar a la espera del ciclo de ejecución del recolector de basura, Java provee la interfaz `AutoCloseable` junto con la estructura de control `try-with-resources` [2]. Este mecanismo asegura que el método `close()` se ejecute de manera implícita e inmediata al salir del bloque de código, garantizando la liberación determinista del recurso antes de que el objeto sea procesado por la memoria [2].

Respecto a la manipulación y almacenamiento de datos. La interfaz `Map`, implementada mediante la clase `HashMap`, permite almacenar elementos en pares clave-valor [3]. Esta estructura resulta adecuada para contabilizar frecuencias, pues permite buscar y actualizar la presencia de una clave con una complejidad promedio de tiempo constante [3]. Adicionalmente, cuando se requiere presentar los datos bajo un orden específico, la clase `TreeSet` permite almacenar elementos únicos manteniendo de forma explícita un ordenamiento alfabético o natural [4].

Finalmente, el procesamiento de cadenas se apoya en el uso de expresiones regulares manejadas mediante la clase `Pattern` y los métodos de la clase `String` [5]. La sustitución de caracteres a través de expresiones regulares permite limpiar las cadenas de entrada removiendo signos de puntuación antes de separar el texto en elementos individuales o tokens [5].

3. Desarrollo

3.1. Actividad 1: Análisis comparativo del concepto de destructor mediante LLM

Inicialmente, se llevó a cabo un ejercicio con distintos modelos de lenguaje para evaluar cómo explican los destructores en Java y las alternativas existentes para la gestión de recursos. Se realizaron tres prompts por distintos integrantes.

3.1.1. Análisis de la respuesta 1

En el primer análisis, ante una consulta directa y sin contexto sobre qué es un destructor en Java, el modelo identificó la premisa errónea, aclaró que Java no posee destructores en sentido estricto y orientó la solución hacia el recolector de basura y el uso de `AutoCloseable`.

El prompt consistió en tres preguntas directas en una sola línea. No proporcionó contexto sobre el nivel de conocimiento previo, el entorno de desarrollo ni restricciones específicas de implementación.

La LLM dividió su respuesta en tres ejes: la gestión automática de memoria mediante el Recolector de Basura (Garbage Collector), la necesidad de liberar recursos externos de forma explícita y las alternativas modernas. También identificó la interfaz `AutoCloseable` junto con la sentencia `try-with-resources` como el equivalente funcional moderno. Proporcionó un ejemplo de código en Java (`ConexionBaseDeDatos`) con implementación de `close()`, su uso dentro de un bloque `try-with-resources` y la salida en consola correspondiente; y mencionó brevemente la obsolescencia del método `finalize()`.

3.1.2. Análisis de la respuesta 2

En la segunda consulta, se solicitó el impacto de un destructor en las clases, la respuesta incorporó diagramas en formato ASCII para ilustrar la pérdida de referen-

cias en la memoria Heap y construyó una tabla comparativa entre las características de C++ y Java.

El prompt se planteó la consulta en un tono informal y conversacional. Solicitó definir qué es un destructor en Java, cómo funciona, cómo afecta a las clases, cómo modifica o cambia las cosas, para qué sirve y un ejemplo práctico.

La respuesta adoptó un estilo conversacional acorde al prompt y asumió explícitamente que el usuario provenía de un entorno con experiencia en C++. Utilizó diagramas textuales en formato ASCII para representar el comportamiento de las referencias a memoria Heap cuando un objeto queda inalcanzable. Además explicó el Garbage Collector, desaconsejó formalmente el uso de `finalize()` por su estado deprecado, presentó la combinación `AutoCloseable` con `try-with-resources` e incluyó una tabla comparativa explícita entre C++ y Java (revisando conceptos como `delete` y `RAII`). Al final dio un ejemplo funcional (`Recurso`) y un esquema del flujo de ejecución.

3.1.3. Análisis de la respuesta 3

En la tercera interacción, donde el planteamiento delimitó explícitamente la diferencia conceptual previa con C++, el modelo entregó un desglose estructurado abordando el comportamiento del recolector de basura como hilo secundario de la JVM, desaconsejó el uso de métodos obsoletos como `finalize()` e introdujo alternativas de limpieza más recientes como las interfaces `Cleaner` y `PhantomReference`.

El integrante aclaró desde el inicio que reconocía que los destructores existen formalmente en C++ y no como tal en Java, pero solicitó la equivalencia correspondiente.

La respuesta presentó una estructura académica y categorizada mediante encabezados, viñetas y tablas formalizadas. Inició con la definición general de destructor en POO. Prosiguió con la explicación del Garbage Collector como hilo secundario de la JVM y dividió los mecanismos de limpieza en Java en estándar moderno (`AutoCloseable` / `try-with-resources`) y mecanismos históricos/avanzados: `finalize()`, `Cleaner`, `PhantomReference`. Al final entregó un ejemplo de código utilizando la interfaz `Closeable`, de la jerarquía de `AutoCloseable`, mostró el bloque `try-with-resources` y sintetizó las diferencias C++ vs Java en una tabla comparativa formal.

3.1.4. Observaciones

El Prompt 1 fue directo pero ingenuo respecto al lenguaje. El Prompt 2 fue conversacional e indagador sobre el impacto en las clases. El Prompt 3 fue el más preciso, delimitando el contexto técnico desde el inicio y exigiendo un desglose de preguntas concretas (qué, cómo, para qué y por qué).

La Respuesta 1 ofreció una solución concisa orientada a la práctica; la Respuesta 2 aportó claridad didáctica mediante diagramas ASCII de memoria; la Respuesta 3 entregó un desarrollo estructurado con cobertura de características avanzadas de Java 9+. Por lo que la Respuesta 3 resultó ser la más adecuada para obtener una descripción

sobre los destructores en java pues el prompt correspondiente guió a la respuesta al establecer el contexto teórico correcto desde el inicio, reconociendo la diferencia entre la destrucción de objetos en C++ y Java. Esto permitió que la respuesta no solo explicara el patrón estándar de liberación de recursos, sino que además desglosara formalmente el funcionamiento del Garbage Collector, la gestión de la memoria Heap y mecanismos avanzados de la JVM como `Cleaner` y `PhantomReference`.

3.2. Actividad 2: Implementación del Analizador de Palabras

La aplicación se organizó en dos clases principales: una clase encargada de la lógica de procesamiento de texto y una clase principal encargada de la interacción con el usuario.

Clase AnalizadorPalabras: Se comienza con el procesamiento de la información, el texto ingresado por el usuario se recibe a través del flujo de entrada estándar e ingresa al constructor de la clase analizadora. Para continuar, la cadena pasa por una etapa de normalización en la cual se convierten todos los caracteres a minúsculas, asegurando que las palabras con variaciones en el uso de mayúsculas se consideren iguales.

En el paso siguiente, se aplica una expresión regular que reemplaza cualquier carácter que no corresponda a una letra del alfabeto, incluyendo vocales acentuadas y la letra "ñ", un número o un espacio en blanco por una cadena vacía. Después de esta limpieza, el texto se fragmenta utilizando una expresión que reconoce uno o más espacios en blanco consecutivos como delimitador, generando un arreglo de términos individuales.

Luego, la lógica recorre secuencialmente el arreglo de palabras e interactúa con un mapa de frecuencias. Si la palabra procesada ya se encuentra registrada como clave dentro de la estructura, se recupera su conteo actual y se incrementa en una unidad; en caso contrario, se registra la palabra por primera vez con un valor inicial de uno.

Posteriormente, para determinar el número total de palabras duplicadas, se itera sobre el conjunto de claves almacenadas en el mapa y evalúa los valores asociados. Cada clave cuyo valor sea estrictamente mayor a uno incrementa un contador general de duplicados.

A continuación, para la visualización de las palabras repetidas, se evalúa una bandera booleana. Si se requiere un resultado desordenado, se recorren las claves del mapa imprimiendo directamente en formato de tabla únicamente aquellos términos con frecuencia mayor a uno. Si se solicita un resultado ordenado, las palabras duplicadas se transfieren a un conjunto de tipo ordenado, el cual las organiza alfabéticamente antes de desplegarlas en la consola.

Clase Main: Para comenzar con el flujo de interacción, se crea e instancia un objeto de lectura de la clase **Scanner** ligado a la entrada estándar del sistema. A continuación, se muestra en la consola un mensaje que solicita al usuario ingresar la oración que se desea analizar, la cual es capturada mediante la lectura de la línea completa y almacenada en una variable de tipo cadena.

Posteriormente, la clase principal instancia un objeto de la clase **AnalizadorPalabras**, pasando como argumento en su constructor la cadena de texto recién ingresada. En

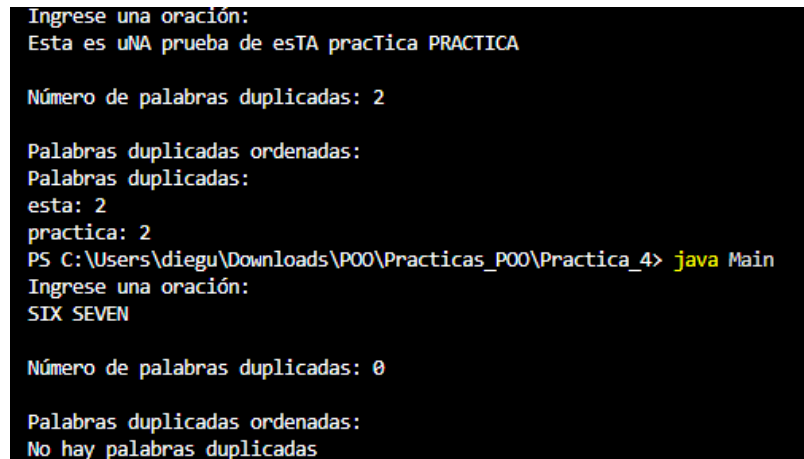
seguida, se realiza la llamada al método de conteo de palabras, recibiendo la estructura de mapa con las frecuencias de cada término y procediendo a imprimir en consola la representación completa del mapa de frecuencias.

Después de mostrar el diccionario general, se invoca el método encargado de calcular el total de palabras duplicadas, almacenando el entero devuelto para desplegar en pantalla el conteo final de palabras distintas que se repitieron. Para continuar, la clase principal ejecuta el método de visualización de duplicados pasando como parámetro el valor lógico falso, lo que genera en la consola la tabla de términos repetidos con sus frecuencias respetando el orden interno de la estructura.

Posteriormente, se efectúa una segunda llamada al método de visualización, transmitiendo esta vez el valor lógico verdadero como argumento para forzar el filtrado y ordenamiento alfabético de los términos duplicados antes de mostrarlos en pantalla. La ejecución concluye cerrando el objeto lector para liberar el flujo de entrada estándar, finalizando así el ciclo de vida del programa.

Finalmente, se realizaron las pruebas de ejecución en el entorno del laboratorio utilizando diversos casos de prueba. Se ingresaron oraciones con repeticiones directas como "niño niño", frases con alternancia estricta de mayúsculas y minúsculas como "Esta es una prueba de esta practica", así como textos que contenían dígitos numéricos y el carácter "ñ". En todos los casos, el programa eliminó los signos de puntuación, convirtió el texto a minúsculas y calculó la frecuencia exacta y el número total de palabras duplicadas.

4. Resultados



```
Ingresa una oración:
Esta es uNA prueba de esTA pracTica PRACTICA

Número de palabras duplicadas: 2

Palabras duplicadas ordenadas:
Palabras duplicadas:
esta: 2
practica: 2
PS C:\Users\diegu\Downloads\P00\Practicas_P00\Practica_4> java Main
Ingresa una oración:
SIX SEVEN

Número de palabras duplicadas: 0

Palabras duplicadas ordenadas:
No hay palabras duplicadas
```

Figura 1: Ingresamos una cadena de texto Esta es uNA prueba de esTA pracTica PRACTICA y regresa las palabras duplicadas mientras que al ingresar SIX SEVEN nos regresa que no hay palabras duplicadas y el número de palabras duplicadas en este caso 0

En el siguiente enlace se encuentra disponible el archivo con el modelo gráfico de la clase main:

[Ver imagen Diagrama Flujo 01 Main.png en GitHub.](#)

- En el diagrama se muestra para ingresar una cadena y llamar a la clase Analizador Palabras

En el siguiente enlace se encuentra disponible el archivo con el modelo gráfico Contar Palabras: [Ver imagen Diagrama Flujo 02 ContarPalabras.png en GitHub.](#)

- En el diagrama se muestra como cuenta palabras con los token y ve si esta vacio despues con un contador muestra la frecuencia.

En el siguiente enlace se encuentra disponible el archivo con el modelo gráfico ObtenerNumerosDuplicados: [Ver imagen Diagrama Flujo 03 obtenerNumerosDuplicados.png en GitHub.](#)

- Obtener Numeros duplicados muestrala la frecuencia que se lleva acabo de la oración dependiendo del contador o suma al contador

En el siguiente enlace se encuentra disponible el archivo con el modelo gráfico ObtenerNumerosDuplicados: [Ver imagen Diagrama Flujo 04 mostrarDuplicados.png en GitHub.](#)

- Dependiendo de las llaves utilizadosy la frecuencia de las palabras repetidas regresa si hay repetidas e imprime las que se repiten o si no hay mas palabras repetidas

5. Conclusiones

Los resultados obtenidos demuestran que la combinación de expresiones regulares con la estructura de datos `HashMap` resuelve la identificación de palabras duplicadas sin importar las variaciones de formato en la entrada. Al transformar el texto a minúsculas y filtrar los caracteres ajenos al alfabeto mediante expresiones regulares, se garantiza que la comparación de cadenas sea uniforme antes de la inserción en el mapa.

Por último, el análisis de las respuestas de los modelos de lenguaje sobre el “destructor” en Java confirmó la importancia de diferenciar entre la memoria administrada automáticamente por el Garbage Collector y la liberación explícita de recursos del sistema mediante `AutoCloseable` y `try-with-resources`.

Referencias

- [1] Oracle (2021). *Garbage Collection Basics*. Java Platform, Standard Edition HotSpot Garbage Collection Tuning Guide. Disponible en: <https://docs.oracle.com/en/java/javase/17/gctuning/introduction-garbage-collector-performance.html>

- [2] Oracle (2021). *AutoCloseable (Java Platform SE 17)*. Oracle Documentation. Disponible en: <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/AutoCloseable.html>
- [3] Oracle (2021). *HashMap (Java Platform SE 17)*. Oracle Documentation. Disponible en: <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/HashMap.html>
- [4] Oracle (2021). *TreeSet (Java Platform SE 17)*. Oracle Documentation. Disponible en: <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/TreeSet.html>
- [5] Oracle (2021). *Pattern (Java Platform SE 17)*. Oracle Documentation. Disponible en: <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/regex/Pattern.html>